

First In, First Out

Week 5 — Queues, Circular Buffers, Deques, and Priority

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 5

Today's Four Questions

1. Why does a stack ruin a print queue?
2. How does one index calculation reclaim wasted array slots?
3. What can a deque do that a queue cannot — and what does it cost?
4. Where does the most urgent job wait when everything is an array?

Plan

Name the queue contract. Fix its array with circular arithmetic. Open both ends, then rank by priority. Close with the Unit-I review.

Handoff: What Week 4 Finished

- ▶ Week 4 gave us conversion: infix to postfix with one left-to-right stack pass, infix to prefix by reverse–swap–postfix–reverse, and prefix to postfix right-to-left (Labs P3, P4) — all $O(n)$ worst case.
- ▶ It priced recursion: Hanoi needs $2^n - 1 = \Theta(2^n)$ moves but only $O(n)$ live frames on the runtime stack.
- ▶ What it never did: hold items in *arrival* order. Every Week 4 structure is last-in, first-out.

This week's job

Give the calculator a waiting room: jobs leave in the order they arrived, the buffer never walks off the end of its array, urgent jobs jump the line — honestly priced (Labs P5, P6).

The Calculator Pipeline, Extended

- ▶ Weeks 3–4 built an expression processor: tokenize, check brackets, convert, evaluate. One program learning new tricks each week.
- ▶ New deficiency: two users hit “print” at once. The second job must *wait its turn* — a stack would serve the newcomer first.
- ▶ The same need appears everywhere: keyboard buffers, process scheduling, server request queues, and (Week 12) the BFS frontier.

The pattern (design principle)

Every structure in this course answers a deficiency the previous one just exhibited. The stack's deficiency is its exit order — the queue is the answer.

One evolving program, not twelve disconnected topics.

Where This Week Sits

- ▶ Week 2 priced things: name the operation, give the bound, state the case. Every claim today repeats that shape.
- ▶ Week 4's close promised: “what changes when the first element out is the first one in — circular buffers, dequeues, and priority.”
- ▶ Week 6 re-implements this week's contract on linked lists; Week 7 compares the two; Week 12 spends a whole traversal inside a queue.

The arc again: state the contract, implement it once, analyse what it costs.

First: why arrival order needs its own contract.

Section 1

The Queue Contract

Why a Stack Ruins a Print Queue

- ▶ Jobs arrive: Alice, Bob, Carol. Fairness means Alice prints first — she arrived first.
- ▶ Push all three on a stack and pop: Carol prints first. The last arrival jumps the line; Alice starves while newcomers keep coming.
- ▶ **Wrong exit order is not a bug you can patch** — LIFO cannot be tuned into FIFO. The contract itself must change.

The requirement in one line

Insert at one end, remove at the other: first in, first out.

The Queue ADT

Definition (Karumanchi Ch. 5, pp.205–223 PDF; Sedgewick Sec. 1.3, p. 120)
[,]

A **queue** is a collection with one removal rule: remove the least recently inserted item. enqueue adds at the **rear**; dequeue removes from the **front**; front peeks without removing. Empty dequeue is an error (the queue exception).

- ▶ Sedgewick's framing: bag, queue, and stack differ *only* in which item leaves next — same contract shape as Week 3's stack.
- ▶ New symbols today: front (next to dequeue), rear (last enqueued) — both array indices, 0-based.

Linear Array Implementation (First Attempt)

Operations on $Q[0..n-1]$, capacity n

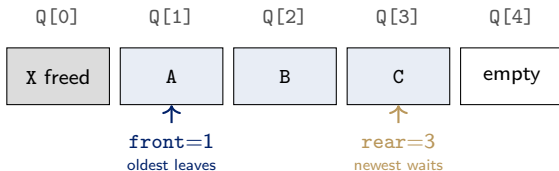
```
enqueue(x): rear = rear + 1; Q[rear] = x
```

```
dequeue(): x = Q[front]; front = front + 1; return x
```

```
empty when front > rear; start with front = 0, rear = -1
```

- ▶ Each operation touches one slot: $O(1)$ worst case — same habit as the stack's $O(1)$ push/pop.
- ▶ But front only marches right. Watch what happens to the slots it leaves behind.

A Linear Queue, Drawn



- X was dequeued; its slot sits empty while `front` marches away from it. The queue holds 3 items but owns 5 slots.

Worked Trace: Enqueue, Enqueue, Dequeue

operation	front	rear	contents
start	0	-1	empty
enqueue(A)	0	0	[A]
enqueue(B)	0	1	[A B]
enqueue(C)	0	2	[A B C]
dequeue $\rightarrow$ A	1	2	[B C]
enqueue(D)	1	3	[B C D]

- Read the dequeue row: the returned value is $Q[\text{front}]$ *before* the increment — off-by-one here returns the wrong job.
- After five operations the live items sit at indices 1–3; index 0 is dead space the implementation cannot reuse.

The False-Full Trap

- ▶ Keep dequeuing and enqueueing: `front` and `rear` both drift right until `rear = n - 1` with live items fewer than n .
- ▶ The implementation reports **full** — but slots 0 to `front-1` stand empty. Overflow with free space: a false alarm.
- ▶ The naive fix (shift everything left on each dequeue) restores honesty at $O(n)$ worst case per dequeue — paying linear time to defend a constant-time contract.

The real fix (next section)

Let the indices wrap around the end of the array instead of marching off it: one modular addition, no shifting, $O(1)$ kept.

Check Your Prediction

Capacity 4: enqueue A, B, C, D, then dequeue twice, then enqueue E. Where does E land?

A: index 0 (dead slot reused) B: index 4 (overflow!) C: index 2

1. A — the linear code never looks left, so it cannot reuse index 0.
2. B — rear was 3, so $\text{rear}+1 = 4$: past the end.
3. C — the queue holds 3 live items, so E takes the third slot.

Trace `front/rear` as numbers, not pictures: B is what the first-attempt code actually does.

The trap is set. Now the one-line arithmetic that springs it.

Section 2

Circular Queue

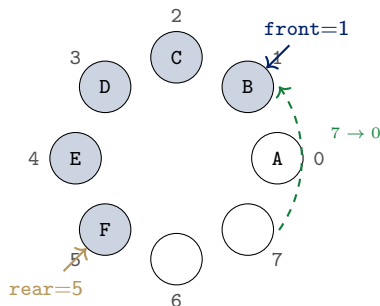
Wraparound Arithmetic

The rule (Karumanchi Ch. 5, pp.205–223 PDF) []

Advance every index modulo the capacity: $\text{rear} = (\text{rear} + 1) \bmod n$, $\text{front} = (\text{front} + 1) \bmod n$. Index $n - 1$ is followed by index 0 — the array is used as a ring.

- ▶ Concrete: capacity 8, $\text{rear} = 7 \rightarrow \text{next rear} = (7 + 1) \bmod 8 = 0$. The “past the end” slot is the freed slot at the start.
- ▶ Cost of the fix: one division remainder per operation — still $O(1)$ worst case. No shifting, ever.

A Circular Queue, Drawn



- Five live items (B–F), `front = 1`, `rear = 5`; the next enqueue lands at index 6, and slot 0 was already recycled once.

Full or Empty? They Look Identical

- ▶ With wraparound, $\text{front} = \text{rear}$ after the first enqueue *and* when the ring is completely full — one equality, two meanings.
- ▶ **Fix A (count)**: keep count of live items; empty when $\text{count} = 0$, full when $\text{count} = n$. One extra integer.
- ▶ **Fix B (sacrificed slot)**: declare full when $(\text{rear} + 1) \bmod n = \text{front}$; the ring holds at most $n - 1$ items and one slot always stands empty.

Course convention (Labs P5)

Use the count version: it holds all n items and its tests read plainly. Name your choice on paper — examiners check the test, not the fashion.

Worked Trace: Wrapping Around (Capacity 4, Count Version)

operation	front	rear	count
enq A, enq B, enq C	0	2	3
deq $\rightarrow$ A, deq $\rightarrow$ B	2	2	1
enq D: $(2 + 1) \bmod 4 = 3$	2	3	2
enq E: $(3 + 1) \bmod 4 = 0$	2	0	3
enq F: $(0 + 1) \bmod 4 = 1$	2	1	4 = full

- ▶ Watch rear walk $2 \rightarrow 3 \rightarrow 0 \rightarrow 1$: the wrap steps are where the linear code died, and the modular code does not even pause.
- ▶ Full test fires honestly: $\text{count} = 4 = n$ with every slot genuinely occupied.

Circular Queue: Cost and Code Shape (Lab P5)

- ▶ enqueue, dequeue, front: each one array access plus one mod — $O(1)$ worst case, $O(1)$ auxiliary space beyond the n -slot array ($n = \text{capacity}$ here).
- ▶ The ring is the week's load-bearing idea: the same wraparound servings keyboard buffers, streaming windows, and Week 12's BFS frontier.
- ▶ Lab P5 implements exactly this trace: circular enqueue / dequeue / display with the full and empty tests above.

The habit (Week 2 shape)

Name the operation (index advance), give the bound ($O(1)$), state the case (worst).
“Circular is $O(1)$ ” without the case is half an answer.

Both ends served by one rule. Now open both ends.

Section 3

Double-Ended Queue

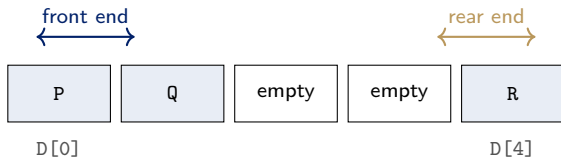
The Double-Ended Queue

Definition (Karumanchi Ch. 5, pp.205–223 PDF; standard treatment) []

A **deque** (“deck”) allows enqueue and dequeue at *both* ends: enqueueFront, enqueueRear, dequeueFront, dequeueRear. A queue is a deque with two of the four operations switched off.

- ▶ Motivation: an undo history takes new states at the rear but drops the *oldest* states from the front when full — one structure, both ends working.
- ▶ Array version: a circular ring with front allowed to step *backwards*: $\text{front} = (\text{front} - 1 + n) \bmod n$.

A Deque, Drawn



insert or remove at either end: all $O(1)$ worst case

- The ring wraps underneath (as in Section 2); the new freedom is that front steps either direction, and so does rear.

Worked Example and Restricted Cousins

- ▶ Start empty (capacity 5): `enqueueRear(Q)`, `enqueueRear(R)` → rear grows right; `enqueueFront(P)` → front steps left with wraparound. State above: `[P Q _ _ R]` with both arrows live.
- ▶ **Input-restricted deque**: insert at one end only, remove at either — models a priority lane that admits newcomers singly.
- ▶ **Output-restricted deque**: insert at either end, remove at one end only — models a pipeline with a single outlet.

Exam note

If a question says “insert at rear, delete at both ends”: that is the input-restricted deque. Name it, then trace it — the name earns the first mark, the trace the rest.

Order served. Now urgency: not who waited longest, but who matters most.

Section 4

Priority Queue Using Arrays

When Arrival Order Is the Wrong Order

- ▶ The OS scheduler holds processes (job, priority): a system interrupt (priority 0) must run before a background backup (priority 9), however late it arrived. FIFO would run the backup first.
- ▶ Hospital triage, printer “urgent” flags, Dijkstra’s frontier (Week 12): every one ranks by **priority**, not by arrival time.
- ▶ Syllabus constraint: implement it *using arrays* — no heaps yet (heaps arrive with trees in Week 11). Two honest array designs, opposite bills.

Definition (Karumanchi Ch. 7, pp.369–408 PDF; Sedgewick Sec. 2.4, p. 308)
[1, 2]

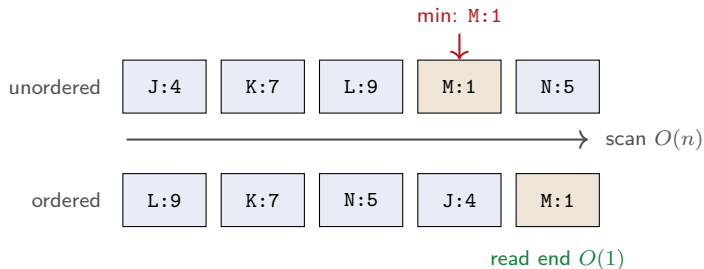
A **priority queue** returns the minimum (or maximum) priority item first. Insert is enqueue with a priority; removal is deleteMin (or deleteMax).

Two Array Designs, Opposite Bills

design	insert	deleteMin
Unordered (append)	$O(1)$ worst case	scan all n : $O(n)$ worst
Ordered (sorted)	shift room: $O(n)$ worst	read off end: $O(1)$ worst

- ▶ No free lunch with arrays: one end of the trade is always linear. (Week 11's heap makes both $O(\log n)$ — the reason heaps exist.)
- ▶ Choose by workload: many inserts, few removals $\rightarrow$ unordered; removals dominate $\rightarrow$ ordered.

Priority Queue, Drawn and Traced



- Trace `deleteMin` above: scan five priorities, return M:1, close the gap by shifting (unordered) or take the end slot (ordered). Lab P6 implements the unordered version.

Check Your Prediction

1 000 inserts then 3 removals: which array design? 3 inserts then 1 000 removals?

A: ordered for both B: unordered, then ordered C: unordered for both

1. A — sorting once up front always wins.
2. B — pay the linear bill on the rare operation, keep the frequent one constant.
3. C — insertion order never matters.

Price the workload, not the structure: count which operation runs hot, then read the table two slides back.

Five weeks of tools. Now the map that holds them together.

Unit I in One Map (Class Test 1)

week	contract	load-bearing skill
1	ADT; malloc/free, NULL	ownership tracing
2	$T(n)$, $O/\Omega/\Theta$, best/worst/average	price every claim
3	Stack ADT; top = -1 empty; $O(1)$	bracket check, postfix eval
4	Notations; conversion; recursion; Hanoi $\Theta(2^n)$	stack traces
5	Queue; front/rear; mod ring	buffer traces (today)

- ▶ The test rewards the course habit: name the structure, trace it line by line, state operation + bound + case.
- ▶ Every structure so far lives in an array; Week 6 asks what pointers buy us instead.

Three Common Mistakes (All of Unit I)

- ▶ **Confusing the case with the bound** — “best case $O(n)$ ” mixes the input (case) with the growth class (bound). Say “best case $\Theta(1)$, worst case $\Theta(n)$ ” with the operation named.
- ▶ **Testing full as `front = rear`** — that test also fires on empty. Use count (0 vs. n) or sacrifice a slot, and write the test down before the code.
- ▶ **Forgetting direction** — stacks pop the young end, queues the old end, dequeues either end, priority queues the urgent end. State the removal rule first; the code follows it.

How to revise

Re-trace one table per week (Weeks 3, 4, and today's two) with a covered answer column — the test is traces, not definitions.

Week 5 Summary

- ▶ **Queue**: enqueue at rear, dequeue from front; $O(1)$ worst case each; the linear array dies of false fullness.
- ▶ **Circular queue**: $(\cdot + 1) \bmod n$ recycling; full vs. empty by count or sacrificed slot; Lab P5.
- ▶ **Deque**: four operations, both ends, backward-stepping front; restricted cousins name which end is closed.
- ▶ **Priority queue on arrays**: unordered ($O(1)$ insert, $O(n)$ deleteMin) vs. ordered (flipped); Lab P6; heaps in Week 11.

Next week

Linked lists: what changes when nodes carry their own links instead of sitting in numbered slots — and the queue reborn on pointers (Week 7).

Before You Go: Predict (Class Test 1 Warm-Up)

Three one-line predictions

Answer from the algorithms, then check in the labs (P5, P6).

1. Capacity 5, `front` = 3, `rear` = 1, `count` = 4: full or not? After one dequeue and one enqueue, what are the three values?
2. Name the deque variant: insert rear-only, remove either end.
3. Unordered priority queue holds A:3, B:1, C:2: `deleteMin` returns what, and what does it cost worst case?

Answers: not full ($4 < 5$); then `front` = 4, `rear` = 2, `count` = 4. Input-restricted deque. Returns B:1 at $O(n)$ worst case.

References I



Narasimha Karumanchi.

Data Structures And Algorithms Made Easy.

CareerMonk Publications, 2017.

This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4th edition, 2011.